Eye Diagram Contour Modeling Using Multilayer Perceptron Neural Networks With Adaptive Sampling and Feature Selection

Chan Hong Goay, Azniza Abd Aziz, *Senior Member, IEEE*, Nur Syazreen Ahmad, *Member, IEEE*, and Patrick Goh

***Abstract*— This article presents a methodology for the modeling of high-speed systems using machine learning methods. A multilayer perceptron neural network is used to map the input–output characteristics from the design parameters to the contours of the eye diagram. In addition, an improved adaptive sampling method is applied for the effective exploration of the design space, and feature selection techniques along with self-organizing maps are used to reduce the problem dimension size. Numerical examples indicate that the proposed method is able to capture the shape and magnitude of the eye contours accurately, and the iterative nature of the algorithm allows a control to balance between accuracy and model generation time. Since well-trained neural networks are able to produce subsequent results almost instantaneously, this modeling approach would be an attractive alternative compared with traditional simulation processes involving complex electromagnetic analyses and long transient simulations.**

***Index Terms*— Eye diagram, feature selection (FS), multilayer perceptron (MLP), self-organizing maps (SOMs), signal integrity (SI).**

I. Introduction

With the increase in the data rates of high-speed circuits, signal integrity (SI) analysis has become more and more challenging for designers and engineers alike. This need for higher data rates results in increasingly complex high-speed circuits with complicated simulation processes, which are very expensive in terms of time and computational power requirements. Normally, SI analysis requires two simulation levels: electromagnetic (EM) field solvers and circuit system simulations [1]. The EM field solver is used to extract the frequency response of the circuit during frequency-domain (FD) simulations, usually in the form of S-parameters. The S-parameters are then used as a broadband representation of the original circuit in the time-domain (TD) simulations to generate the output voltage and current waveforms for various excitations. These output waveforms are usually constructed in the form of an eye diagram, for statistical SI analysis of the high-speed system. In general, EM field solvers are very accurate but are also very slow. Even though circuit simulators are comparatively much faster, when the simulation involves very long bit sequences, it can still take a very long time. In addition, multiple simulations will often have to be performed during the design process covering several sets of design parameters. Effects of manufacturing limitations that cause variations in the design parameters also have to be considered, as these variations can affect the electrical performances of the circuits. These challenges complicate the circuit design process in an industry where short time-to-market is an inherent requirement and the first-time pass is becoming the expected norm. Thus, there is an ever-increasing demand for faster and more efficient strategies for high-speed circuit modeling and analysis.

Recently, machine learning (ML) methods have gained popularity as a new approach for modeling and analyzing complicated circuit and structures. ML methods can learn from the simulation data of a modeled problem, and then perform prediction based on the design-parameter (inputs)-to-electrical-property (outputs) relationship they learned. The underlying appeal of ML methods is that once they are properly trained, they can model the problem many times faster than the EM or circuit solvers. Gaussian processes (GPs) and Bayesian optimization (BO) are examples of ML methods what have been used for SI optimization and error detections with great success [2], [3]. Support-vector machines (SVMs) have also been applied to perform uncertainty qualification and performance assessment of high-speed circuits [4]–[6]. On the other end of the spectrum, neural network-based techniques have also seen a resurgence and proven to be reliable over the years, with applications from RF and microwave circuit modeling [7]–[10] to SI applications [11]–[16]. Compared with the nonparametric methods, neural networks often have lower computational costs when the number of training samples is large and does not require the selection of a proper kernel. However, neural networks can be susceptible to overfitting and the hyperparameters of the neural networks should be carefully chosen to avoid that. In addition, when the sample distribution changes, the choice of hyperparameters needs to be reevaluated for optimal performance.

Data collection is the first step in the ML model development, and undoubtedly a very important step. Insufficient

Manuscript received May 5, 2019; revised August 23, 2019; accepted August 27, 2019. Date of publication August 30, 2019; date of current version December 6, 2019. This work was supported by Universiti Sains Malaysia through the Research University (RU) Grant: 1001/PELECT/8014011. Recommended for publication by Associate Editor W. T. Beyene upon evaluation of reviewers' comments. *(Corresponding author: Patrick Goh.)*

The authors are with the School of Electrical and Electronic Engineering, Universiti Sains Malaysia, Nibong Tebal 14300, Malaysia (e-mail: eepatrick@usm.my).

Color versions of one or more of the figures in this article are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TCPMT.2019.2938583

2156-3950 © 2019 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

Authorized licensed use limited to: TRINITY COLLEGE LIBRARY DUBLIN. Downloaded on March 09,2026 at 14:30:17 UTC from IEEE Xplore. Restrictions apply.

samples or undersampling can cause incomplete design space exploration, whereas excessive training or oversampling is unnecessary and can add additional cost to the already expensive ML model development process. Traditionally, one-shot sampling methods such as design of experiment (DoE) and uniform grid sampling are used in the data set generation process for ML circuit modeling. Usually, the designers do not have the information about the level of nonlinearity across the design space before performing the simulation process. In addition, the level of nonlinearity is also not typically uniform across the entire design space. In other words, if the design space were to be divided into many smaller regions, every region would have a different degree of nonlinearity. Thus, it is very easy for one-shot design approaches to cause undersampling in nonlinear regions and oversampling in highly linear regions especially when a uniform grid sampling approach is used. If the sampling method fails to provide the desired exploration of the design space after the sampling process, then the designers may have to reset the grid or even repeat the sampling process with another sampling method, which ultimately leads to waste of time and resources, since the generation of each sample can be very costly. In addition, it is very difficult to explore the whole design space manually as the design space is often too large and each simulation can take a very long time to complete. Therefore, alternative sampling strategies such as the adaptive sampling have been proposed, where they generate data points iteratively and intelligently at locations with high nonlinearity to provide the greatest information gain [17]. However, the adaptive sampling strategy often focuses too hard on a few small, highly nonlinear regions, causing it to get stuck in these regions. This usually happens when these highly nonlinear regions also contain erroneous data. Sometimes, this can also prevent the ML method from converging. Thus, this article presents a modification on the original adaptive sampling algorithm, allowing it to escape from these regions.

Eye diagram characteristics such as eye height (EH) and eye width (EW) are important metrics for SI evaluation. Thus, most of the past work for ML modeling focuses on EH and EW predictions [6], [18], [19]. In this article, we propose a new modeling strategy to model the entire eye contour instead of just the EH and the EW. To overcome the added complexity of such modeling, adaptive sampling strategies are adopted for accurate design space exploration. Self-organizing maps (SOMs) are then used to group designs according to their similarities and feature extraction techniques are used to extract the key features of the modeled eye diagrams. This allows a dimensional reduction in the number of input samples and reduces the associated training costs. Artificial neural networks (ANNs) are then used to learn the relationship between the input samples and the eye contour. Numerical results are presented, which illustrate the performance and viability of the proposed method.

II. ANN-Based Signal Integrity Modeling

A. Artificial Neural Networks

An ANN is a simplified mathematical model of a biological neural network that consists of interconnected neurons.

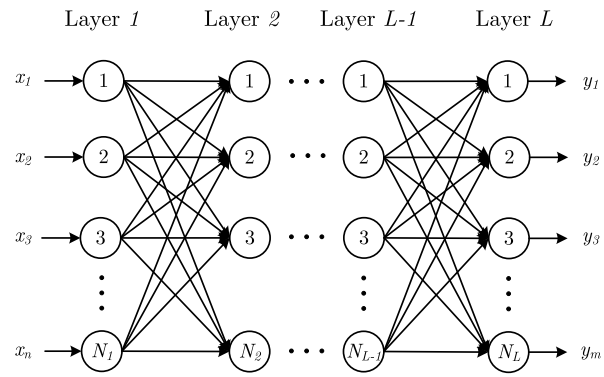

Fig. 1. General MLP structure.

The approximation capability of an ANN is most succinctly explained by the universal approximation theorem, which states that a feedforward neural network with a finite number of neurons in a single hidden layer can approximate any nonlinear multidimensional function to any degree of accuracy [20]. The multilayer perceptron (MLP) is one of the most widely used ANN structures. Its neurons are arranged in L layers, where layer 1 is the input layer, layer L is the output layer, and layers 2 to $L-1$ are hidden layers. A neural network with one hidden layer is often considered a shallow neural network, while those with multiple hidden layers are considered deep neural networks. The general structure of an L-layer perceptron with n input neurons and m output neurons is shown in Fig. 1. During training, ANNs learn by adjusting its weights as to minimize the training errors, which are defined as the differences between the desired outputs from the training samples and the modeled outputs by the ANN. The training goal is to achieve generalization. A neural model with good generalization can provide accurate answers even when it is tested with inputs that it has never encountered before in the training process. Early stopping and regularization are often used to improve the generalization capability of a neural network and avoid overfitting. The readers are referred to [21] and [22] for more background about the ANNs.

MLPs are a class of feedforward neural networks where the computations are done in a single direction from the external inputs, x_i to the outputs, y_i. The feedforward computation from the input layer to the output layer is summarized in the following equations:

$$\text{Input Layer: } z_i^1 = x_i, \quad i = 1, 2, \ldots, N_1, n = N_1 \tag{1}$$

$$\text{Hidden Layer: } z_i^l = \sigma\left(\gamma_i^l\right), \quad i = 1, 2, \ldots, N_l, \quad l = 2, 3, \ldots, L \tag{2}$$

$$\text{Output Layer: } z_i^L = y_i, \quad i = 1, 2, \ldots, N_L, m = N_L \tag{3}$$

where N_1 is the number of input neurons, N_L is the number of output neurons, and N_l is the number of hidden neurons at the lth layer. Fig. 2 shows the structure of the ith hidden neuron at the lth layer. Each neuron is connected to all outputs of the previous layer, and each connection between the ith neuron at the lth layer and the jth neuron at the $(l-1)$th layer is assigned a weight w_{ij}^l. Besides that, each neuron also has an additional weight parameter, w_{i0}^l, which represents the bias. To obtain the output of a neuron, the weighted sum of the inputs, γ_i^l, is

Authorized licensed use limited to: TRINITY COLLEGE LIBRARY DUBLIN. Downloaded on March 09,2026 at 14:30:17 UTC from IEEE Xplore. Restrictions apply.

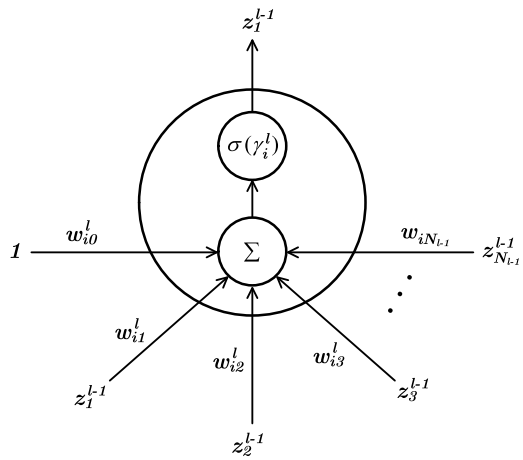

Fig. 2. Structure of the ith hidden neuron of the lth layer.

first computed. Then, the output of the neuron, z_i^l, is obtained by applying an activation function, σ, to γ_i^l. These activation functions are smooth switching functions that decide when a neuron should fire. Common activation functions include the sigmoid function

$$\sigma\left(\gamma_i^l\right) = \frac{1}{1 + e^{-\gamma_i^l}} \tag{4}$$

and the tansig function

$$\sigma\left(\gamma_i^l\right) = \frac{2}{1 + e^{-\gamma_i^l}} - 1 \tag{5}$$

where in each case

$$\gamma_i^l = \sum_{j=0}^{N_{l-1}} w_{ij}^l z_j^{l-1}. \tag{6}$$

Hyperparameter tuning for training neural networks is often required to obtain a reasonable model, and to avoid both underfitting and overfitting. In this article, a coarse-to-fine search algorithm is used to search for the optimal number of hidden neurons during the adaptive sampling process [23]. The coarse-to-fine search algorithm is a modified version of the grid search algorithm. It performs a coarse search initially, searching with a large step size, followed by a fine search that searches around the best hyperparameter value with a much smaller step size. BO [24] also provides an alternative option to perform hyperparameter search and tuning automatically especially when the number of parameters to be tuned is large.

Levenberg–Marquardt backpropagation is used to train the neural networks [25]. The Levenberg–Marquardt algorithm uses the following approximation to the Hessian matrix in a Newton-like update:

$$w_{k+1} = w_k - [J^T J + \mu I]^{-1} J^T e \tag{7}$$

where J is the Jacobian matrix that contains first derivatives of the network errors with respect to the weights and biases and e is a vector of network errors. The initial values of μ and the increment and decrement factors are fixed to be 1, 1.5, and 0.8, respectively, to prevent the training from converging too rapidly since early stopping is used to improve generalization.

B. Sampling Techniques

In the past, DoE has been used in SI neural modeling applications for data generation. In [26], DoE-based sets of high-speed channel parameters are used to generate the FD and TD responses, which are then used to generate training data for a neural model that predicts EH and EW from S-parameters. In [27], a variant of DoE called the Taguchi DoE, which uses orthogonal arrays for efficient design space exploration, is used for training set generation of a neural model that predicts EH and timing jitter from the design parameters of high-speed interconnects. Although DoE has been widely used in a lot of applications in the past, it is a one-shot design approach, which has some drawbacks. One-shot approaches usually require information about the linearity of the problem to minimize oversampling or undersampling. If the designers do not have a full understanding of the input-target mapping function, then it is very difficult to decide on the sample size to be generated. In addition, one-shot design approaches usually separate the data generation and training processes, making them less ideal as the sampling methods of choice for fully automated neural model generations.

In [18], adaptive sampling is used as the sampling method for SI neural modeling applications because it allows better integration between the data generation and training processes. The adaptive sampling algorithm begins by separating the whole design space into 2^d regions with equal volume, where d is the number of design parameters. Sometimes, the algorithm can also begin with only a single region. The adaptive sampling algorithm then generates a minimum amount of samples uniformly across the regions. Intermediate neural models are created using currently available data to determine the linearity of each region. At the end of each iteration, the region with the largest validation error will be identified as the worst performing region, and it will be split into 2^d new regions. The sampling process will be terminated once the accuracy of the intermediate neural model meets the user-defined goal. Fig. 3 shows the splitting of the worst performing region during the adaptive sampling process for a 2-D modeling problem. In this case, the training points are generated at the corners of the regions, whereas the validation points are generated at the center of the regions. Once the splitting is completed, the validation point of the worst performing region becomes a training point in the next iteration. The pseudocode of the adaptive sampling algorithm is presented in Algorithm 1.

Adaptive sampling has been used for automated model generation (AMG), which integrates the data generation and the neural model training into a single process [28]–[31]. This reduces the amount of human-based manual modeling and monitoring during the development of the neural model.

C. Eye Diagram Prediction

Eye diagram prediction is one of the most commonly seen applications of neural networks in the field of SI. Specifically, most of the work focuses on modeling the EHs and the EWs [6], [12], [18], [19], [26], [32]. This is because the minimum heights and widths of signals at the receiver are important metrics for the performance evaluation of

Authorized licensed use limited to: TRINITY COLLEGE LIBRARY DUBLIN. Downloaded on March 09,2026 at 14:30:17 UTC from IEEE Xplore. Restrictions apply.

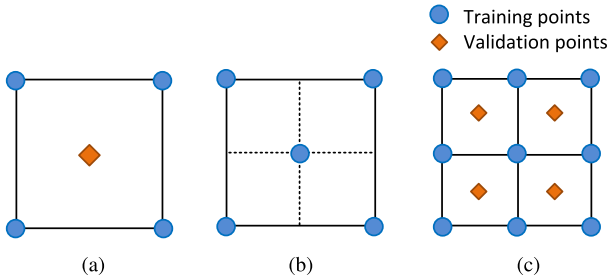

Fig. 3. 2-D visualization of the splitting process. (a) Worst performing region. (b) Region is split into 2^d new regions, four new regions in this case; also the validation point at the center of the original region is changed to a training point. (c) New training and validation points are generated for the worst performing region.

Algorithm 1 Adaptive Sampling

```
Initialize design space as region R
while (netPerformance < desiredPerformance) do
    Generate training and validation samples within R
    Create and train an intermediate neural model using training samples
    Compute the network's performance using validation samples
    if (netPerformance < desiredPerformance) then
        Identify worst performing region as R*
        Split R* into equal volume regions R_1, R_2, ..., R_k, where k = 2^d
        Remove R* from R
        Add regions R_1, R_2, ..., R_k into R where R = [R, [R_1, R_2, ..., R_k]]
    end if
end while
```

high-speed channels. Sometimes, a neural network is also used to model the timing jitter of the eye [27]. These problems can be described briefly as follows. Suppose that there is a function that maps design parameters (inputs) to eye metrics (outputs) and that a neural network is to be used to learn that function. Conventionally, two simulations are carried out to get the eye diagram. First, frequency responses such as S-parameters are extracted from the design. Then, the S-parameters are used as a representation of the design in a transient simulation to get the output waveform and the eye diagram is constructed. By using a trained neural network as a replacement for this mapping, both simulations can be omitted, speeding up the design process significantly. This is especially useful in the cases where the eye diagrams need to be generated repeatedly, as the design parameters are tweaked, for example, during an optimization process.

III. Improved Adaptive Sampling Algorithm

The adaptive sampling method is driven by the fact that the training data generation process is computationally very expensive. Ideal training data generation requires that the samples be generated at optimal positions to explore the design space properly with only a small amount of samples. In this article, the adaptive sampling algorithm is used in the data generation process, where it focuses on generating data at nonlinear regions iteratively. Let R_i be the ith region of a list of regions, R, where R_i is in the form of

$$R_i = \begin{bmatrix} B_{i,1} \\ B_{i,2} \\ B_{i,3} \\ B_{i,4} \\ \vdots \\ B_{i,(2j-1)} \\ B_{i,2j} \\ \vdots \\ B_{i,(2d-1)} \\ B_{i,2d} \end{bmatrix} \tag{8}$$

where $B_{i,(2j-1)}$ and $B_{i,2j}$ are the minimum and maximum values of the jth design parameter in the ith region. Also

$$R_i \in R; \quad R = [R_1, R_2, \ldots, R_{i-1}, R_i, R_{i+1}, \ldots]. \tag{9}$$

If R_i is the worst performing region, then it will be split into 2^d equal volume regions at the end of the iteration and R_i becomes a vector of 2^d smaller regions named R_{new}

$$R_i \xrightarrow{\text{split}} R_{\text{new}} \tag{10}$$

where

$$R_{\text{new}} = [R_{\text{new},1}, R_{\text{new},2}, R_{\text{new},3}, \ldots, R_{\text{new},2^d}]. \tag{11}$$

After that, R_i will be removed from R and the R_{new}'s will be added to R. R now becomes

$$R_i \notin R; R = [R_1, R_2, \ldots, R_{i-1}, R_{\text{new}}, R_{i+1}, \ldots]. \tag{12}$$

The sampling distribution used for the generation of samples within regions can be any distribution defined by the user. 2K-DoE generates samples at the corner of a region [33]. Star distribution generates samples at the center of each face of the region, called star points, and at the center of the region. Rotated star distribution generates samples called rotated star points, whose locations are the elements of the product of the star points and a rotational matrix [34]. For a d-dimensional region, the number of samples generated by 2K-DoE, star distribution, and rotated star distribution is 2^d, $(2d + 1)$, and $2d$, respectively.

Although the number of samples can increase substantially with the number of dimensions, the effect can be mitigated by carefully considering the sampling mechanism in conjunction with the modeled problem. In this article, 2K-DoE with a center point and star distribution without a center point are initially used for the training and validation samples, which will generate $(2^d + 1)$ and $2d$ samples, respectively. Then, in the next iteration, the 2K-DoE method will generate training points that overlap with the validation points generated by the star points in the last iteration. Thus, the validation points can be reused as training points in the next iteration. This can be visualized in Fig. 4. In addition, the design parameters can be grouped into two groups, which are EM (geometrical parameters of the physical channel) and non-EM (termination resistance, parasitic capacitance, equalization parameters,

Authorized licensed use limited to: TRINITY COLLEGE LIBRARY DUBLIN. Downloaded on March 09,2026 at 14:30:17 UTC from IEEE Xplore. Restrictions apply.

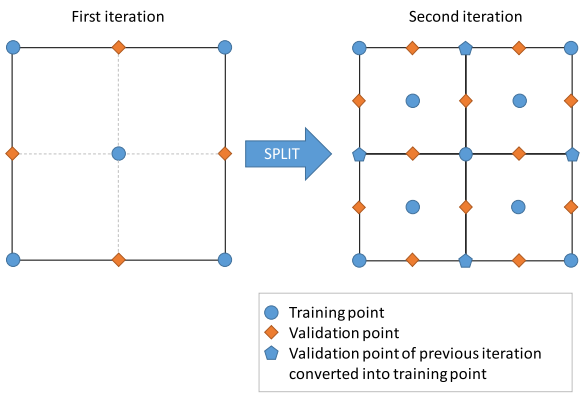

Fig. 4. Sample distribution for the first and second iterations during the adaptive sampling process for a 2-D modeling problem.

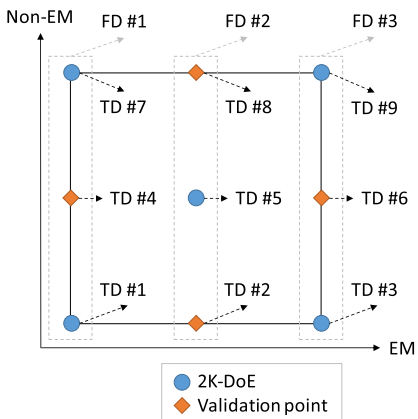

Fig. 5. Exploration of a 2-D design space consisting of an EM and a non-EM design parameter using corner points plus a center point, and star points. This requires three FD simulations.

and so on) design parameters. If multiple samples share the same EM design parameters, only one FD simulation is performed. Each sample still requires one TD simulation, but that is usually much faster than the FD simulation. Fig. 5 shows that for a 2-D modeling problem, with one EM design parameter and one non-EM design parameter, with nine samples (training samples are corner points plus a center point and validation samples are star points), only three FD simulations are required instead of nine. Note that if rotated star points are used instead of star points, five FD simulations would have been required, as shown in Fig. 6.

A. Weakness of the Original Adaptive Sampling Algorithm

Adaptive sampling is a very efficient and reliable sampling technique, but it is not without any weaknesses. The main problem with adaptive sampling is that it can be "trapped" inside a highly nonlinear region. One reason for this is that it might be due to erroneous simulations or extractions. If this happens, the algorithm will always try to generate samples around that region while completely ignoring other regions. Then, a large amount of erroneous data are used for the neural model development, which will result in an unusable neural model. Sometimes, a region behaves differently from the rest of the regions. For example, small changes in design parameters can lead to extreme changes in electrical properties.

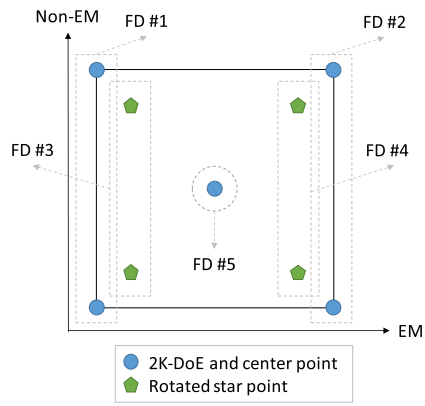

Fig. 6. Exploration of a 2-D design space consisting of an EM and a non-EM design parameter using corner points plus a center point, and rotated star points. This requires five FD simulations.

This can also cause the algorithm to be "trapped" inside that region. This is because the original adaptive sampling identifies the worst performing region solely based on the validation errors and it does not consider the volume of a region. A large region with no samples can lead to high uncertainty of its linearity.

B. Modifications to the Adaptive Sampling Algorithm

The adaptive sampling algorithm is improved to include the effect of the region volume during the selection of the worst performing region. A term named region error, E_r, is introduced and defined as follows:

$$E_r = \frac{\mathrm{mse}(y_{N,r} - t_{N,r})}{n_r} \tag{13}$$

where $\mathrm{mse}(y_{N,r} - t_{N,r})$ is the mean squared error between the normalized outputs of the neural model $y_{N,r}$, and the corresponding normalized desired outputs $t_{N,r}$, respectively, and n_r is the weight given to a region r according to its volume. The value of n_r can be computed as follows:

$$n_r = \log_{2^d}\left(\frac{\text{volume of entire design space}}{\text{volume of region } r}\right) + 1 \tag{14}$$

$$\text{volume of region } r = \prod_{i}^{n}(x_{\max,i} - x_{\min,i}), \qquad x_i \in r \tag{15}$$

where $x_{\max,i}$ and $x_{\min,i}$ are the maximum and minimum values of the ith design parameter or the ith ANN input in the region r. E_r is then used to select the worst performing region. Each time an intermediate neural model is created, the E_r values of every region are computed using the validation points. The region with the largest E_r value will be selected as the worst performing region, R^*.

Fig. 7 shows the values of n_r for every region of the design space of a 2-D problem at the beginning and the end of the adaptive sampling algorithm. The original region, which covers the whole design space, has the n_r value of 1. Splitting this region will produce 2^d new regions with $1/2^d$ of the original volume, and the n_r value will be increased by 1. Thus, a smaller region will have a higher n_r value, and

Authorized licensed use limited to: TRINITY COLLEGE LIBRARY DUBLIN. Downloaded on March 09,2026 at 14:30:17 UTC from IEEE Xplore. Restrictions apply.

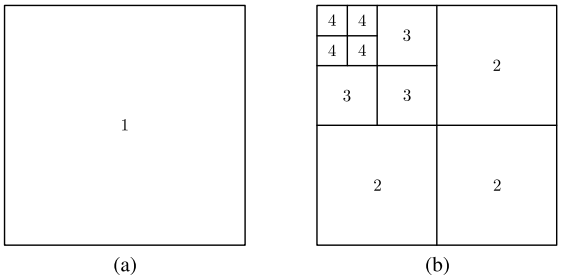

Fig. 7. Values of n_r for every region in the design space of a 2-D problem (a) before splitting and (b) after being split three times.

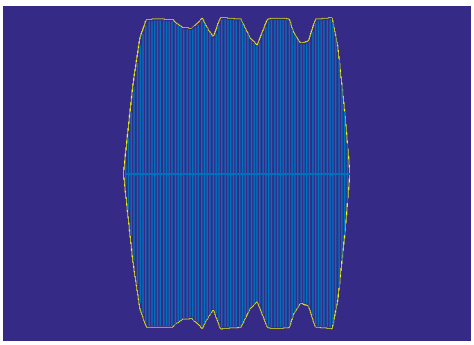

Fig. 8. 81 EHs across one UI.

the higher n_r value will act to lower the value of E_r. This lowers the chance of this region for it to be identified as the worst performing region. This simple modification allows the adaptive sampling algorithm to split the region more evenly in terms of volume, thus preventing it from being "trapped."

IV. Prediction of Eye Contour

In this article, we propose a new modeling approach, where the entire eye contour is modeled. The aim of doing this is to improve the flexibility of the outputs beyond just obtaining the EH and EW. Instead of just taking a sample at the center of the eye, a certain amount of samples is collected across one unit interval (UI) at predefined time points and the EHs are modeled as a function of time, EH(t). Fig. 8 shows an example, where 81 EHs are recorded in a 2.5-Gb/s system, and Fig. 9 shows a plot of its EH(t) versus t. For a horizontally symmetrical eye, its eye contour can be reconstructed from EH(t) by

$$\text{Eye Contour} = \begin{cases} +\dfrac{1}{2}EH(t) \text{ for upper half} \\ -\dfrac{1}{2}EH(t) \text{ for lower half.} \end{cases} \tag{16}$$

A reconstruction of the eye contour is shown in Fig. 10. The EW is not recorded since it can already be obtained from the eye contour.

Fig. 11 compares the ANN topologies between the modeling of the EH/EW and the eye contour. For the modeling of EH and EW, the ANN has d inputs, which means that the ANN models a d-dimensional problem. By contrast, the ANN for the modeling of eye contour has time, t, as one additional input and the EW is not modeled at the outputs. Thus, the modeled problem is a $(d+1)$-dimensional problem. If the EHs at certain

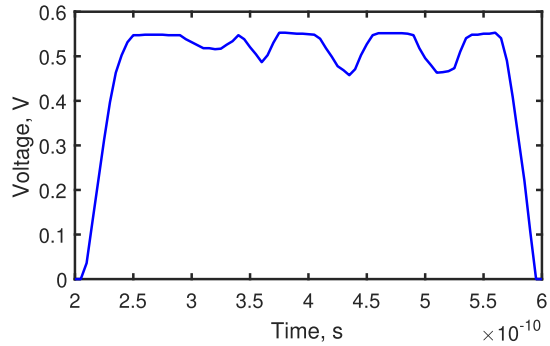

Fig. 9. Plot of EH(t) versus t.

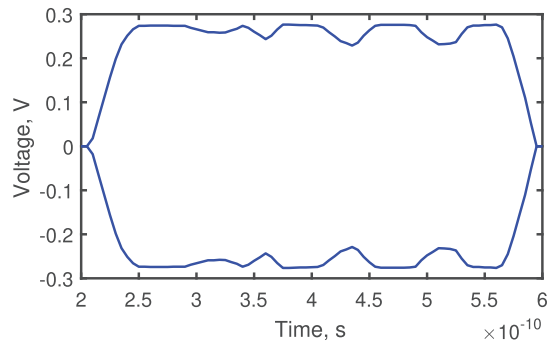

Fig. 10. Reconstructed eye contour from EH(t).

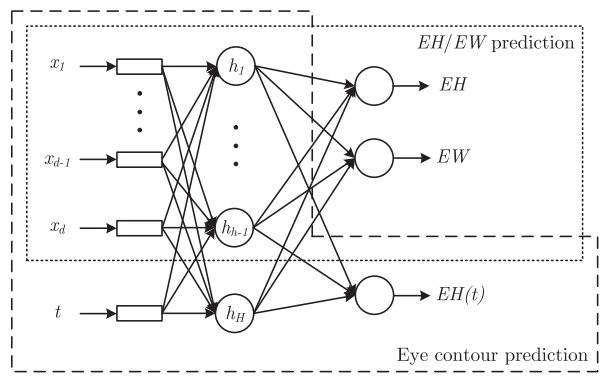

Fig. 11. Comparison of the ANN topologies for EH/EW and eye contour predictions.

time points of each design are recorded and the number of time points per eye contour is N_t, the length of the training data would become N_t times larger than the previous modeling approach even though both have the same number of designs. This will slow down the training time of the ANN. Thus, a method is needed to alleviate this problem. In this article, feature selection (FS) is used to reduce the size of N_t. This is discussed in Section V. Note that time t is not considered to be one of the design parameters of the design space. Thus, t is excluded during the region splitting and the region error computation in the adaptive sampling algorithm.

A. Feature Selection Techniques

In general, FS techniques can be grouped into two major approaches: classifier-dependent (wrapper and embedded

Authorized licensed use limited to: TRINITY COLLEGE LIBRARY DUBLIN. Downloaded on March 09,2026 at 14:30:17 UTC from IEEE Xplore. Restrictions apply.

methods) and classifier-independent (filter method). In this article, the filter method is used. Usually, filters are computationally less expensive and less prone to overfitting. Filters separate FS and classifier components and use specific scoring criteria to rank the features. A good FS technique should ensure feature relevance and interfeature correlation.

For a distribution X, the entropy of distribution $H(X)$ is defined as [35]

$$H(X) = -\sum_{x \in X} p(x) \log p(x) \tag{17}$$

which is the amount of uncertainty in X. The conditional entropy of X given Y is

$$H(X|Y) = -\sum_{y \in Y} p(y) \sum_{x \in X} p(x|y) \log p(x|y) \tag{18}$$

where $p(x)$ and $p(y)$ are the marginal probability density functions and $p(x|y)$ is the conditional probability density function. Then, the mutual information between X and Y is defined as

$$\begin{aligned} I(X;Y) &= H(X) - H(X|Y) \\ &= \sum_{x \in X} \sum_{y \in Y} p(xy) \log \frac{p(xy)}{p(x)p(y)} \end{aligned} \tag{19}$$

and the conditional mutual information is

$$\begin{aligned} I(X;Y|Z) &= H(X|Z) - H(X|YZ) \\ &= \sum_{z \in Z} p(z) \sum_{x \in X} \sum_{y \in Y} p(xy|z) \log \frac{p(xy|z)}{p(x|z)p(y|z)}. \end{aligned} \tag{20}$$

Filters use specific scoring criteria for feature ranking. Each filter method has its own criterion, which is used to measure the correlation between a feature and the class label. In general, a stronger correlation between a feature and the class label indicates that the feature is highly useful when it is used in the classifier. There are many different FS methods that are reviewed briefly here [35]–[37]. For a class label Y, the mutual information maximization (MIM) criterion of a feature X_k is defined as

$$J_{\mathrm{MIM}}(X_k) = I(X_k;Y) \tag{21}$$

where it measures the correlation between the feature and the class label. The mutual information FS (MIFS) criterion is defined as

$$J_{\mathrm{MIFS}}(X_k) = I(X_k;Y) - \beta \sum_{X_j \in S} I(X_k;X_j) \tag{22}$$

where S is the set of currently selected features, and β is a configurable parameter. The term $I(X_k;Y)$ ensures feature relevance, and the subsequent term represents the penalty to ensure low correlations with the selected features in S. The joint mutual information (JMI) criterion is defined as

$$J_{\mathrm{JMI}}(X_k) = \sum_{X_j \in S} I(X_k X_j;Y) \tag{23}$$

while the max-relevance min-redundancy (MRMR) criterion is defined as

$$J_{\mathrm{MRMR}}(X_k) = I(X_k;Y) - \frac{1}{|S|} \sum_{X_j \in S} I(X_k;X_j). \tag{24}$$

The conditional MIM (CMIM) criterion is defined as

$$J_{\mathrm{CMIM}}(X_k) = \min_{X_j \in S} [I(X_k;Y|X_j)] \tag{25}$$

the interaction capping (ICAP) criterion is defined as

$$\begin{aligned} J_{\mathrm{ICAP}}(X_k) = {} & I(X_k;Y) \\ & - \sum_{X_j \in S} \max[0, \{I(X_k;X_j) - I(X_k;X_j|Y)\}] \end{aligned} \tag{26}$$

and the BetaGamma selection criterion is defined as

$$\begin{aligned} J_{\mathrm{BetaGamna}} = {} & I(X_k;Y) \\ & - \beta \sum_{X_j \in S} I(X_j;X_k) + \gamma \sum_{X_j \in S} I(X_j;X_k|Y) \end{aligned} \tag{27}$$

where, in this case, β is the interfeature dependence penalization factor and γ is the conditional penalization factor. Both of them can be any value between zero and one. The conditional redundancy (ConDred) criterion, J_{ConDred}, is based on (27) with $\beta = 0$ and $\gamma = 1$, whereas the conditional infomax feature extraction (CIFE) criterion, J_{CIFE}, is also based on the same equation but with $\beta = 1$ and $\gamma = 1$. Last, the double-input symmetrical relevance (DISR) criterion is defined as

$$J_{\mathrm{DISR}}(X_k) = \sum_{X_j \in S} \frac{I(X_k X_j;Y)}{H(X_k X_j Y)}. \tag{28}$$

B. Applying Feature Selection on Eye Contour

For the eye contour modeling, the EHs of all designs, EH(t), are stored in a matrix as

$$\begin{bmatrix} EH_1(t_{\min}) & EH_1(t_{\min}+dt) & \dots & EH_1(t_{\max}) \\ EH_2(t_{\min}) & EH_2(t_{\min}+dt) & \dots & EH_2(t_{\max}) \\ \vdots & \vdots & \ddots & \vdots \\ EH_{N_d}(t_{\min}) & EH_{N_d}(t_{\min}+dt) & \dots & EH_{N_d}(t_{\max}) \end{bmatrix} \tag{29}$$

where the time points have a range of $t_{\min}$ to $t_{\max}$ with a step of dt, and N_d is the number of designs. The EH matrix is then binned to 500 levels. The binned EH matrix, BEH(t), is then fed into an SOM in order to assign the designs into different groups. An SOM is a type of ANN that is used to classify a large amount of data according to the similarities between the data. The main advantage of an SOM is that it is trained using unsupervised learning, which means that it can learn to form its own classification without any external help [38], and can then be used to label any subsequent data. In this article, labeling the designs according their contributions to the magnitude of EHs, such as that in [19], can be problematic because it is not just the individual values that are important, but also the variations among the values that form the

Authorized licensed use limited to: TRINITY COLLEGE LIBRARY DUBLIN. Downloaded on March 09,2026 at 14:30:17 UTC from IEEE Xplore. Restrictions apply.

eye contour. Thus, if the data are labeled based on the magnitude of the EHs, points toward the middle of the eye, which normally carry larger values, might be prioritized more during the FS process. In addition, the relationship between the design parameters and the shape of the BER contour is unknown before the simulations are performed. Thus, the SOM is used to label the EHs of every designs based on their similarities and grouped from one to nine. Then, the FS process is carried out based on BEH(t), the labels, the number of features to be selected, and the FS method. The FS methods used in this article are MRMR, CMIM, JMI, DISR, CIFE, ICAP, and ConDred. The FS process starts with a low number of features to be selected, and slowly increases it iteratively. The extracted features of all the designs are then used in a 1-D data interpolation process to obtain the values at the time points of unselected features. The interpolated data are then compared with the original data and the coefficient of determination, R^2, of all designs is computed, where R^2 is defined in general as

$$R^2 = 1 - \frac{\sum(y - \hat{y})^2}{\sum(y - \overline{y})^2} \tag{30}$$

where y and $\hat{y}$ are the two responses to be compared, the original EHs and interpolated EHs from the selected features in this case, and $\overline{y}$ is the mean of y.

The FS process will stop if the goal is met in terms of R^2. In addition, the eye-crossing points of all the designs before the start of the FS process are recorded, and at the end of the FS process, if an eye-crossing point is not one of the selected features, it would be autoincluded as a selected feature. This is because, usually, only the left and right eye-crossing points have values of zero out of all the features. Thus, both of them contain essential information that can affect the shape of the eye contour. Other than that, R^2 will also be computed for each nonselected feature at the end of each iteration of the adaptive sampling algorithm. The top three nonselected features with the worst interpolation performance will be identified and stored in a list, together with all the top three features from the past iterations. The members in this list will be included in the selected features at the end of the next iteration if they are not selected during the FS process. This is a correction mechanism to ensure that the nonselected features that give large interpolation errors in the current iteration will be selected in the next iteration. The pseudocode for implementing the FS method is presented in Algorithm 2.

The FS process is demonstrated using the EHs of 76 designs. Before performing the FS, the number of initial features must be determined. Having too many initial features can cause a high redundancy in the data, whereas having too few initial features can increase the number of iterations the algorithm needs to meet its goal. An experiment is performed with one initial feature. The number of selected features is then increased by one until the FS methods select all 81 features, and the result is shown in Fig. 12. Based on the graph, any number from 15 to 25 can be a reasonable number of initial features, and in this article, 20 initial features are used. In addition, the number of features to be selected will be increased by two if the goal is not met at the end of the iteration to accelerate convergence.

Algorithm 2 FS

```
Define initial number of features as k
Find eye crossing points of EH(t)
while (interpolation performance < goal) do
  BEH(t) = EH(t) binned to 500 levels
  Use SOM to assign labels for every design
  for (i = 1 : number of FS methods) do
    Perform the FS process using one of the FS method
    and the SOM labels, selecting k features
    Combine the features with the eye crossing points and
    the top three worst performing features from the last
    adaptive sampling iteration
    for (j = 1 : number of designs) do
      Interpolate on the unselected features using the
      selected features
      Calculate R^2
    end for
    Calculate the average R^2
  end for
  Increase k by a small number
end while
Select the best FS method and return the features
```

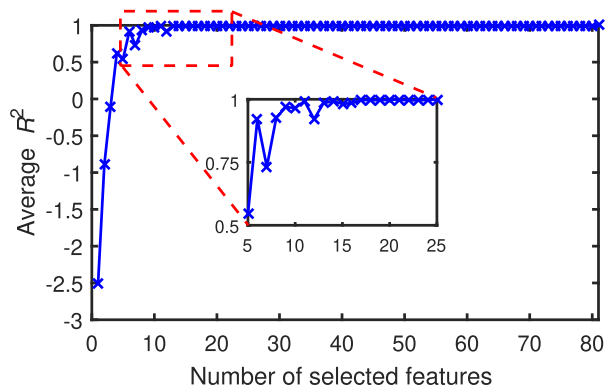

Fig. 12. Average R^2 across all designs versus number of selected features.

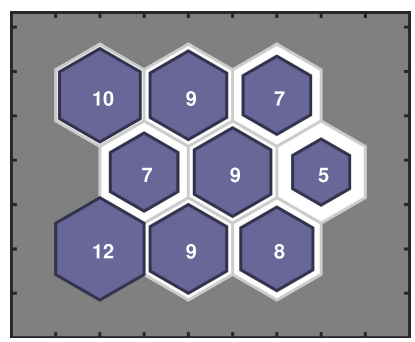

Fig. 13. Sample hits of an SOM.

Fig. 13 shows the SOM with a 3×3 structure and its sample hits after it is fed with the binned EHs of all the designs. As can be seen, all 76 designs are assigned to nine different groups. Fig. 14 shows a comparison of the different EHs of all the designs after the FS process. Originally, the EHs of each design have 81 time points. The number of selected features from all the methods is tabulated in Table I. In the last iteration, seven sets of 32 features are selected, each by

Authorized licensed use limited to: TRINITY COLLEGE LIBRARY DUBLIN. Downloaded on March 09,2026 at 14:30:17 UTC from IEEE Xplore. Restrictions apply.

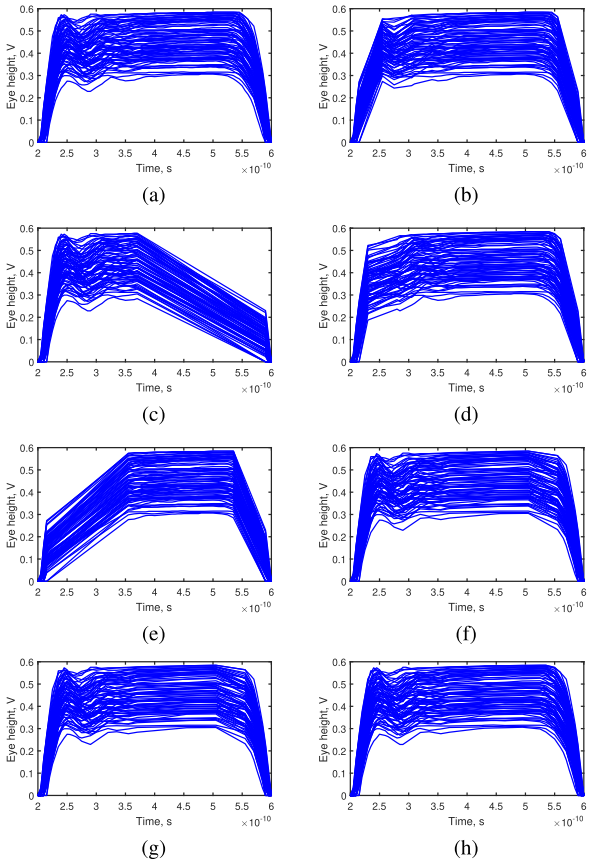

Fig. 14. EHs after FS with methods. (a) None. (b) MRMR. (c) CMIM. (d) JMI. (e) DISR. (f) CIFE. (g) ICAP. (h) ConDred.

TABLE I

NUMBER OF SELECTED FEATURES AND AVERAGE R^2 ACROSS ALL DESIGNS FOR DIFFERENT FS METHODS

Method	Number of Selected Features	Average R^2
None	81	1
MRMR	33	0.9833
CMIM	35	-3.4756
JMI	38	0.9887
DISR	38	0.2339
CIFE	32	0.9983
ICAP	32	0.9983
ConDred	39	0.9994

one of the seven FS methods. Then, the extra features such as the eye-crossing points are added to the features if they have not already been selected by the FS methods. For example, CIFE and ICAP had already selected all the extra features, and thus, no additional points are added to the selected features. Fig. 14(a) shows the original EHs, whereas Fig. 14(b)–(h) shows the EHs after FS with MRMR, CMIM, JMI, DISR, CIFE, ICAP, and ConDred, respectively. In this case, the best performance is obtained with ConDred, where 39 features are selected out of the original 81. All of the FS techniques are implemented using the FEAST Toolbox in MATLAB [35]. The final structure of the proposed ANN is shown in Fig. 15.

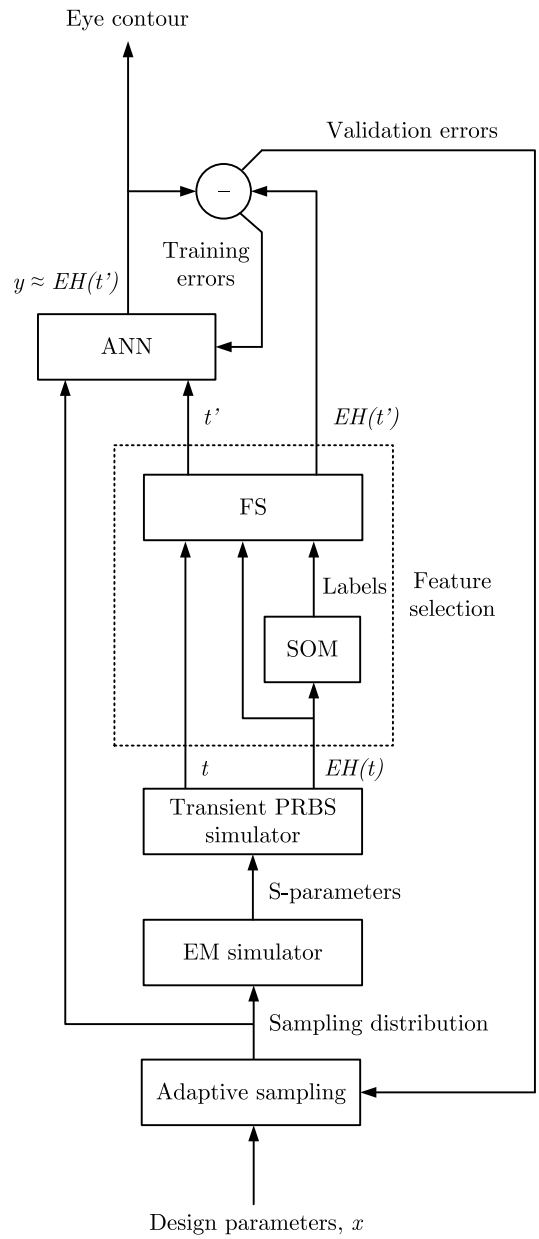

Fig. 15. Structure of the proposed ANN model for eye contour modeling.

V. NUMERICAL RESULTS

In this section, numerical results are presented using the proposed technique. The circuit used in this article is a typical low-voltage differential signaling (LVDS) circuit, as shown in Fig. 16. The transmitter (with an 8B10B encoder) outputs data at a rate of 2.5 GHz, a rise/fall time of 40 ps, and a voltage level of -0.3 to 0.3 V. Each end of the differential microstrip line is terminated with a termination resistor and a parasitic capacitor with capacitance C. Neural networks are used to model the contour of the eye diagram at the receiver. Three examples are shown using the same circuit. The first example is a 3-D modeling problem, while the second example is a 5-D modeling problem. The third example is similar to the second example but with an added dimension from a receiver equalization. The TD simulations are performed using

Authorized licensed use limited to: TRINITY COLLEGE LIBRARY DUBLIN. Downloaded on March 09,2026 at 14:30:17 UTC from IEEE Xplore. Restrictions apply.

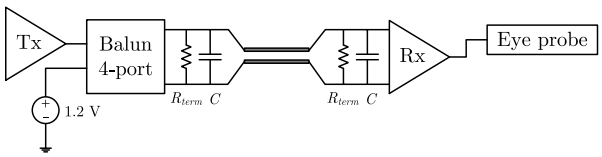

Fig. 16. LVDS circuit for the EH(t) modeling example.

TABLE II
3-D DESIGN SPACE PARAMETERS

Design Parameters	Min Value	Max Value
Line Length, L_{Tline} (mils)	2000	4000
Line Width, W_{Tline} (mils)	20	40
Line Separation, S_{Tline} (mils)	10	40

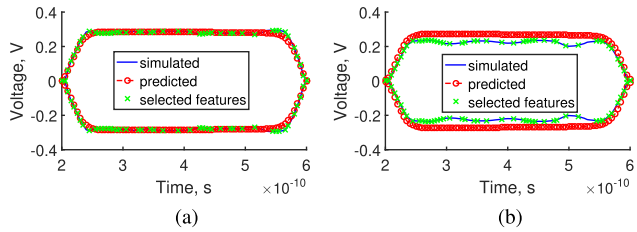

Fig. 17. 3-D EH(t) modeling problem. Actual eye contours and reconstructed eye contours from predictions of neural models of (a) test case 1 and (b) test case 2 in the first iteration.

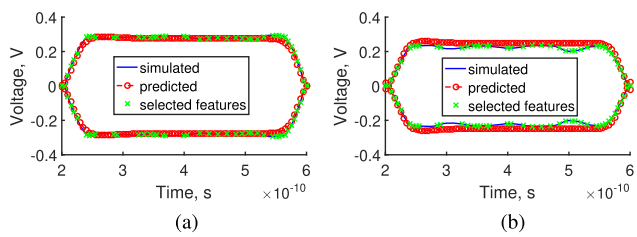

Fig. 18. 3-D EH(t) modeling problem. Actual eye contours and reconstructed eye contours from predictions of neural models of (a) test case 1 and (b) test case 2 in the second iteration.

Keysight's Advanced Design System (ADS) simulator, whereas the 3-D EM or FD simulations are performed using SonnetLite.

A. 3-D Modeling Problem

The design space of this problem is tabulated in Table II. The design constants are the termination resistance R_{term} (100 Ω); relative permittivity ϵ_r (4.4); C (0.05 pF); substrate height, h_{sub} (80 mil); loss tangent, $\tan\delta$ (0.01); metal conductivity (5.8×10^7 S/m), and metal thickness, h_{Tline} (1.4 mil). For each design, the EHs are measured over 81 time points: from 0.2 ns (−0.5 UI) to 0.6 ns (0.5 UI), with a step of 0.005 ns (0.0125 UI). To reduce the number of time points further, the FS method, as discussed in Section IV, is used. Only the EHs of the selected features are used as the training and validation data. An MLP with a single hidden layer is trained to learn the relationship between the input parameters and the eye contour. The training goal is a validation performance (in terms of R^2) $\geq$ 0.98. Fifty additional random designs are used for testing. The whole

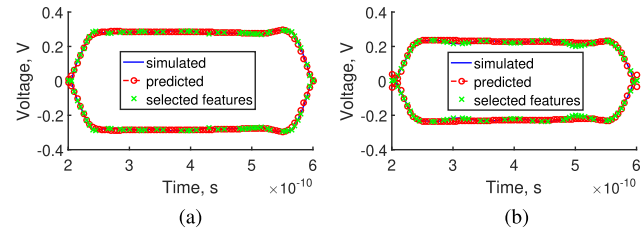

Fig. 19. 3-D EH(t) modeling problem. Actual eye contours and reconstructed eye contours from predictions of neural models of (a) test case 1 and (b) test case 2 in the third iteration.

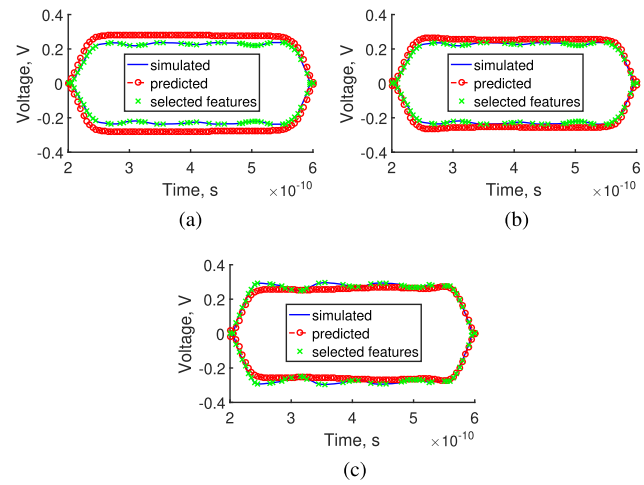

Fig. 20. 3-D EH(t) modeling problem. Actual eye contours and reconstructed eye contours from predictions of neural models of the worst test cases in (a) first, (b) second, and (c) third iterations.

data gathering process took 223 min, and an additional 60 s was used for training the neural models. Once trained, the neural model uses only 0.015 s to model the eye contour of a design, consisting of EHs at 81 time points, whereas it takes about 216 s to complete the simulation process from the S-parameter extraction using the 3-D EM software to the simulation of the eye diagram. This means that although a neural model takes a significant amount of time to develop, the designers can still save a lot of time later during the design process of the modeled circuit or another similar circuit. The results are presented in Table III.

The training process meets the goal in the third iteration. CIFE, ConDred, and JMI were the best performing FS methods during the first, second, and third iterations, respectively, with the number of selected features equal to 46, 43, and 47. Two random test cases from the 50 designs are selected for comparison. Figs. 17–19 show the actual eye contours and the reconstructed eye contours from the predictions of the neural models for the first, second, and third iterations of the adaptive sampling process, respectively. The results show that the ANNs can model the general shape of the eye diagrams even in the first iteration, and the performance slowly improves as more samples are added. The worst cases for the neural models in all the three iterations are shown in Fig. 20. In general, the worst-case error reduces as the number of iterations increases. The distributions of the training and validation samples for all the three iterations are shown in Fig. 21.

Authorized licensed use limited to: TRINITY COLLEGE LIBRARY DUBLIN. Downloaded on March 09,2026 at 14:30:17 UTC from IEEE Xplore. Restrictions apply.

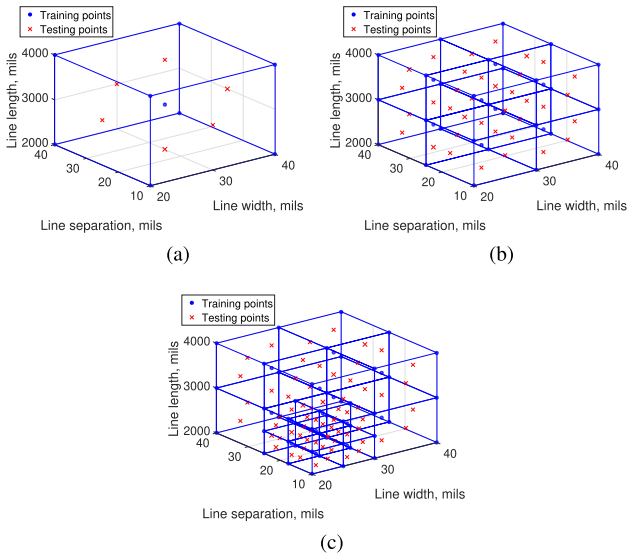

Fig. 21. 3-D EH(t) modeling problem. Sample distributions after (a) first, (b) second, and (c) third iterations.

TABLE III
RESULTS OF THE 3-D EYE CONTOUR MODELING

Performance Parameters	1st Iteration	2nd Iteration	3rd Iteration
No. of Hidden Neurons	4	6	12
No. of Designs for Training	9	35	61
No. of Designs for Validation	6	36	66
No. of Selected Features	46	43	47
Feature Selection Method	CIFE	ConDred	JMI
Training Performance, R^2	0.9815	0.9789	0.9864
Validation Performance, R^2	0.9787	0.9792	0.9830
Testing Performance, R^2	0.9375	0.9602	0.9686
Worst Case Performance, R^2	0.4461	0.8292	0.9112

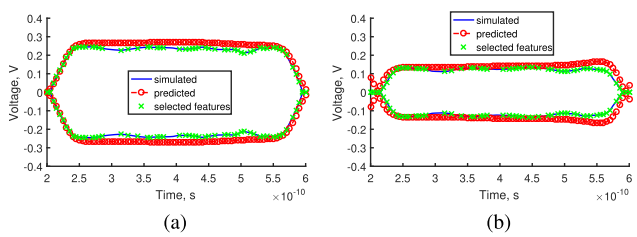

Fig. 22. 5-D EH(t) modeling problem. Actual eye contours and reconstructed eye contours from predictions of neural models of (a) test case 1 and (b) test case 2 in the first iteration.

To provide some perspective on the accuracy of the method used in this article, a comparison is made with other existing methods for the different stages of the modeling process. First, the improved adaptive sampling is compared with a Latin hypercube sampling (LHS) method [39]. We use LHS to generate the same amount of samples (61 training samples and 66 validation samples), train a neural model with 12 hidden neurons, and then test it using the same 50 testing samples. The LHS gave a validation performance of 0.9653, testing the performance of 0.9631, and worst-case performance of 0.7756. This shows that LHS has a comparable testing performance as the adaptive sampling, but a much worse worst-case performance.

TABLE IV
5-D DESIGN SPACE PARAMETERS

Design Parameters	Min Value	Max Value
Termination Resistance, R_{term} (Ω)	100	200
Line Length, L_{Tline} (mils)	2000	4000
Line Width, W_{Tline} (mils)	20	40
Line Separation, S_{Tline} (mils)	10	40
Substrate Thickness, h_{sub} (mils)	80	160

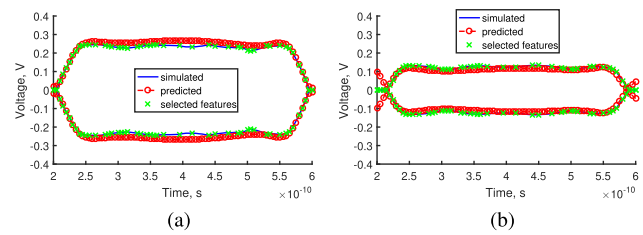

Fig. 23. 5-D EH(t) modeling problem. Actual eye contours and reconstructed eye contours from predictions of neural models of (a) test case 1 and (b) test case 2 in the second iteration.

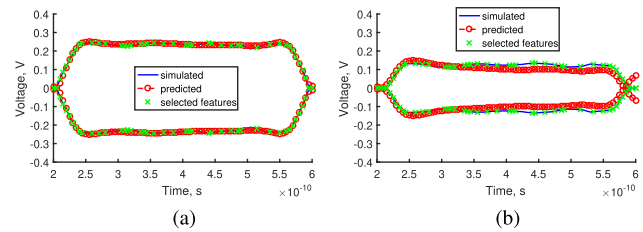

Fig. 24. 5-D EH(t) modeling problem. Actual eye contours and reconstructed eye contours from predictions of neural models of (a) test case 1 and (b) test case 2 in the third iteration.

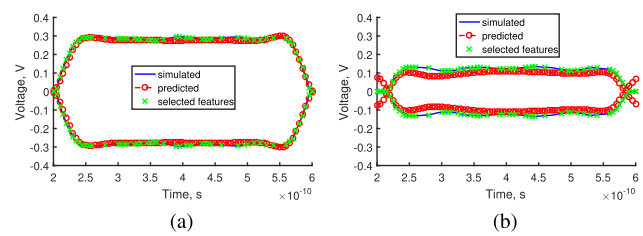

Fig. 25. 5-D EH(t) modeling problem. Actual eye contours and reconstructed eye contours from predictions of neural models of (a) test case 1 and (b) test case 2 in the fourth iteration.

Next, an experiment is conducted to compare the use of autoencoders for feature extraction. The readers are referred to [40] for a background on autoencoders. An autoencoder is trained from EH(t). The encoder maps 81 time points onto a vector with a length of 47. Then, a neural model is trained to predict that vector from the design parameters, and finally, the predictions made by the neural model are converted back to the time points using the decoder. This approach gives a validation performance of 0.9684, testing performance of 0.9581, and worst-case performance of 0.7894, which is not better than the results obtained using FS in this article. Moreover, since the outputs from the encoder are arbitrary and do not represent time points, the neural network outputs can no longer be grouped into one, and it takes

Authorized licensed use limited to: TRINITY COLLEGE LIBRARY DUBLIN. Downloaded on March 09,2026 at 14:30:17 UTC from IEEE Xplore. Restrictions apply.

TABLE V
RESULTS OF THE 5-D EYE CONTOUR MODELING

Performance Parameters	1st Iteration	2nd Iteration	3rd Iteration	4th Iteration	5th Iteration
No. of Hidden Neurons	7	16	62	28	48
No. of Designs for Training	33	275	517	740	980
No. of Designs for Validation	10	240	470	700	930
No. of Selected Features	55	48	44	52	49
Feature Selection Method	ConDred	CMIM	CMIM	CMIM	CMIM
Training Performance, R^2	0.9722	0.9759	0.9868	0.9772	0.9859
Validation Performance, R^2	0.9708	0.9680	0.9763	0.9769	0.9845
Testing Performance, R^2	0.9190	0.9310	0.9253	0.9476	0.9631
Worst Case Performance, R^2	-0.2631	0.1367	0.2970	0.3359	0.4201

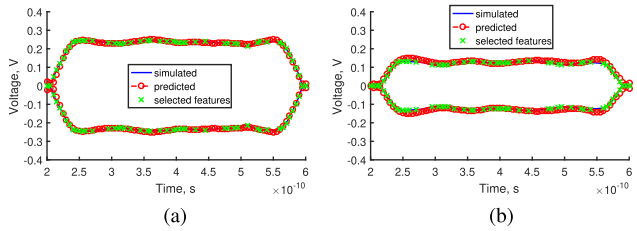

Fig. 26. 5-D EH(t) modeling problem. Actual eye contours and reconstructed eye contours from predictions of neural models of (a) test case 1 and (b) test case 2 in the fifth iteration.

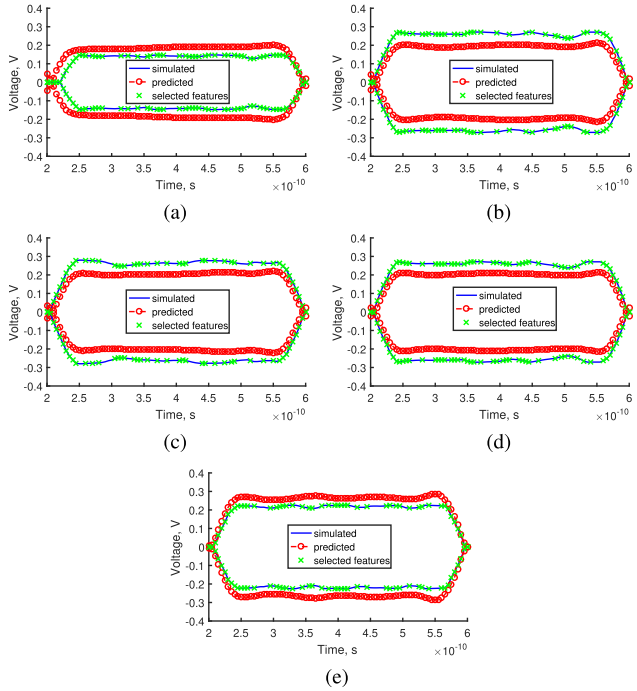

Fig. 27. 5-D EH(t) modeling problem. Actual eye contours and reconstructed eye contours from predictions of neural models of the worst test cases in (a) first, (b) second, (c) third, (d) fourth, and (e) fifth iterations.

about 22.5 times longer to train, compared with the original single output model. However, it is to be remarked that there has been recent progress in variational autoencoders [41], [42], and these could be pursued as a viable dimensionality-reduction alternative in future work.

Finally, we compare BO with the coarse-to-fine search for searching the optimal number of hidden neurons with the same number of iterations for both the methods and using the same

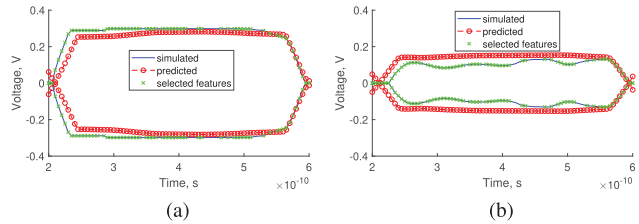

Fig. 28. 6-D EH(t) modeling problem. Actual eye contours and reconstructed eye contours from predictions of neural models of (a) test case 1 and (b) test case 2 in the first iteration.

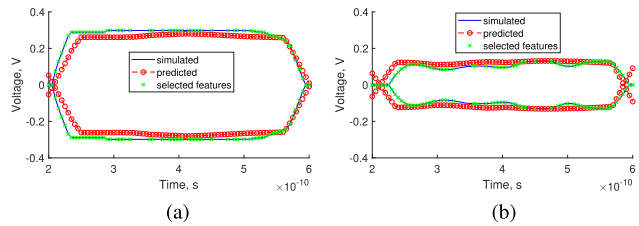

Fig. 29. 6-D EH(t) modeling problem. Actual eye contours and reconstructed eye contours from predictions of neural models of (a) test case 1 and (b) test case 2 in the second iteration.

training, validation, and testing samples in this example. Both show comparable testing performance, 0.9721 for coarse-to-fine search and 0.9701 for BO. The coarse-to-fine search is slightly faster than the BO in this case since we are only optimizing one hyperparameter. If the number of hyperparameters is large, for example, in high-dimensional cases, BO could be a better alternative since the number of possible configurations of hyperparameters in the coarse-to-fine search could become too large.

B. 5-D Modeling Problem

The design space of this problem is tabulated in Table IV. The training goal is a validation performance (in terms of R^2) $\geq$ 0.98; 250 additional random samples are generated for testing the trained neural network. The whole data generation process in this case took 2267 min, and an additional 79 min was used for training the neural models. The results are presented in Table V.

The training process meets the goal in the fifth iteration. ConDred was the best performing FS method in the first iteration, while CMIM was the best performing FS method in the second, third, fourth, and fifth iterations. The numbers of selected features are 55, 48, 44, 52, and 49 for the first to

Authorized licensed use limited to: TRINITY COLLEGE LIBRARY DUBLIN. Downloaded on March 09,2026 at 14:30:17 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
RESULTS OF THE 6-D EYE CONTOUR MODELING

Performance Parameters	1st Iteration	2nd Iteration	3rd Iteration	4th Iteration
No. of Hidden Neurons	17-10-11	10-14-10	11-13-17	20-20-19
No. of Designs for Training	65	629	1271	1732
No. of Designs for Validation	12	588	1164	1708
No. of Selected Features	63	61	52	41
Feature Selection Method	ConDred	JMI	JMI	JMI
Training Performance, R^2	0.9507	0.8857	0.9525	0.9894
Validation Performance, R^2	0.9340	0.9179	0.9249	0.9876
Testing Performance, R^2	0.8717	0.8748	0.9100	0.9883
Worst Case Performance, R^2	0.3412	0.3303	0.4202	0.9095

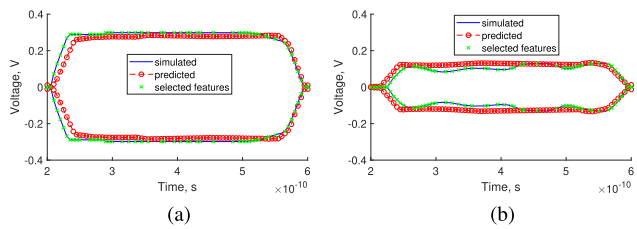

Fig. 30. 6-D EH(t) modeling problem. Actual eye contours and reconstructed eye contours from predictions of neural models of (a) test case 1 and (b) test case 2 in the third iteration.

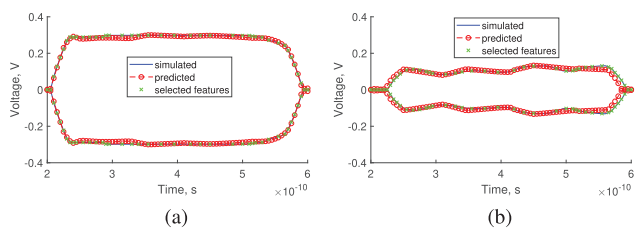

Fig. 31. 6-D EH(t) modeling problem. Actual eye contours and reconstructed eye contours from predictions of neural models of (a) test case 1 and (b) test case 2 in the fourth iteration.

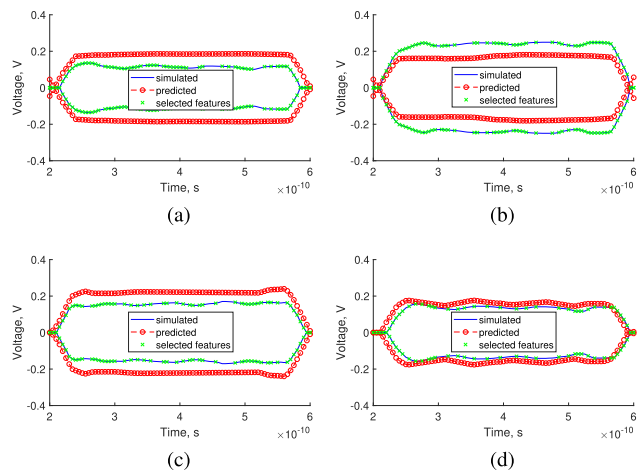

Fig. 32. 6-D EH(t) modeling problem. Actual eye contours and reconstructed eye contours from predictions of neural models of the worst test cases in (a) first, (b) second, (c) third, and (d) fourth iterations.

fifth iteration, respectively. Two random test cases from the 250 designs are selected for comparison. Figs. 22–26 show the actual eye contours and the reconstructed eye contours from the predictions of the neural models in the first to fifth iteration. The results show that the ANNs can already model the general shape of the eye diagrams in the first three iterations and the prediction accuracy improved in the last two iterations, capturing even minute details in the eye contour. The worst cases for the neural models in all five iterations are shown in Fig. 27. If necessary, the worst-case error can be reduced by increasing the number of iterations.

It is to be noted that the model development time can be significantly reduced by reusing data from previously done simulations. For example, in the fifth iteration, to generate a total of 1910 samples, only 670 FD simulations are needed. This is because of the location of the four EM design parameters (L_{Tline}, W_{Tline}, S_{Tline}, and h_{sub}), one non-EM design parameter (R_{term}), and samples at corner points and star points in the design space, which permits the reusability of the FD samples. This is beneficial as the time required for an FD simulation is much longer than a TD simulation. For example, the simulation time needed for an existing design is approximately 25 s, whereas about 216 s is needed for a new design. In addition, the iterative procedure used in this modeling approach allows the performance to be improved by increasing the number of samples during each iteration. This permits a tradeoff between speed and accuracy.

C. 6-D Modeling Problem

In this example, the same design space as in the 5-D example is used with an added decision feedback equalizer (DFE) at the receiver. The number of equalization taps N_{Taps} acts as an additional design parameter, where it ranges from zero to four. An MLP with three hidden layers is used to learn the relationship between the inputs and the outputs in this problem. Hyperparameter searching is performed to find the optimal number of neurons for each hidden layer and the optimal learning rates. Since the number of hyperparameters to be optimized is large, BO is used in this example. The training goal is a validation performance (in terms of R^2) ≥ 0.98. For each region in the adaptive sampling process, only $(2^{d-1} - d)$ points are generated by the 2K-DoE from the second iteration onward in order to reduce the amount of training samples; 500 additional test samples are generated to test the trained neural network. A total of 3440 samples were generated (529 unique EM designs), and it took 3117 min to complete the FD and TD simulations. An additional 190 min was used for training the neural models. The results are presented in Table VI.

Authorized licensed use limited to: TRINITY COLLEGE LIBRARY DUBLIN. Downloaded on March 09,2026 at 14:30:17 UTC from IEEE Xplore. Restrictions apply.

The training process meets the goal in the fourth iteration. ConDred was the best performing FS method in the first iteration, while JMI was the best performing FS method in the second, third, and fourth iterations. The numbers of selected features are 63, 41, 52, and 41 for the first to fourth iteration, respectively. Two test cases from the 500 designs are selected for comparison. Figs. 28–31 show the actual eye contours and the reconstructed eye contours from the predictions of the neural models in the first to fourth iteration, while the worst cases for the neural models in all iterations are shown in Fig. 32.

VI. Conclusion

In this article, ANNs have been applied as a promising alternative to traditional circuit simulation techniques in the field of SI for eye diagram modeling. An improvement to the original adaptive sampling algorithm is presented, which minimizes the likelihood of undersampling and oversampling and FS techniques are used to reduce the dimension of the input. Numerical results indicate that the trained ANN is able to produce accurate eye contour predictions and the accuracy can be improved in an iterative manner by adding more samples. Since a trained ANN is able to compute results instantaneously, the developed model can be reused in subsequent design simulation, analysis, and optimization to replace computationally expensive EM and circuit simulations.

While ML approaches have gained considerable attention due to the increased computing capabilities of modern systems, the significant training cost is still a subject for improvement to accelerate the model development process. As an extension of this article, simpler approximation models such as rational functions, SVMs, and GP regressions can be used in conjunction with ANNs for the nonlinear exploration of the design space as they have lower development costs. This allows the ANN to be created only in the last iteration, and by reducing the number of new samples per iteration, while simultaneously increasing the number of iterations, a smoother and more precise control over the model development process can be obtained. In addition, the development of the modular subblocks of the neural network components that are then unified in a larger system also presents an interesting challenge and opportunity. At the single-ANN level, development of novel neural network structures where outputs of hidden neurons carry physical significance would be very attractive from a design standpoint.

References

[1] S. H. Hall and H. L. Heck, *Advanced Signal Integrity for High-Speed Digital Designs*. Hoboken, NJ, USA: Wiley, 2009.

[2] H. M. Torun and M. Swaminathan, "High-dimensional global optimization method for high-frequency electronic design," *IEEE Trans. Microw. Theory Techn.*, vol. 67, no. 6, pp. 2128–2142, Jun. 2019.

[3] R. Medico, D. Spina, D. V. Ginste, D. Deschrijver, and T. Dhaene, "Machine learning-based error detection and design optimization in signal integrity applications," *IEEE Trans. Compon., Packag., Manuf. Technol.*, to be published.

[4] R. Trinchero, M. Larbi, H. Torun, F. G. Canavero, and M. Swaminathan, "Machine learning and uncertainty quantification for surrogate models of integrated devices with a large number of parameters," *IEEE Access*, vol. 7, pp. 4056–4066, 2019.

[5] R. Trinchero, P. Manfredi, I. S. Stievano, and F. G. Canavero, "Machine learning for the performance assessment of high-speed links," *IEEE Trans. Electromagn. Compat.*, vol. 60, no. 6, pp. 1627–1634, Dec. 2018.

[6] T. Lu, J. Sun, K. Wu, and Z. Yang, "High-speed channel modeling with machine learning methods for signal integrity analysis," *IEEE Trans. Electromagn. Compat.*, vol. 60, no. 6, pp. 1957–1964, Dec. 2018.

[7] Q.-J. Zhang, K. C. Gupta, and V. K. Devabhaktuni, "Artificial neural networks for RF and microwave design—From theory to practice," *IEEE Trans. Microw. Theory Techn.*, vol. 51, no. 4, pp. 1339–1350, Apr. 2003.

[8] J. E. Rayas-Sanchez, "EM-based optimization of microwave circuits using artificial neural networks: The state-of-the-art," *IEEE Trans. Microw. Theory Techn.*, vol. 52, no. 1, pp. 420–435, Jan. 2004.

[9] P. Burrascano, S. Fiori, and M. Mongiardo, "A review of artificial neural networks applications in microwave computer-aided design (invited article)," *Int. J. RF Microw. Comput.-Aided Eng.*, vol. 9, no. 3, pp. 158–174, 1999.

[10] F. Wang, V. K. Devabhaktuni, C. Xi, and Q.-J. Zhang, "Neural network structures and training algorithms for RF and microwave applications," *Int. J. RF Microw. Comput.-Aided Eng.*, vol. 9, no. 3, pp. 216–240, May 1999.

[11] K. Hsu, W. Guo, G. Shiue, C. Lin, T. Huang, and R. Wu, "Design of reflectionless vias using neural network-based approach," *IEEE Trans. Adv. Packag.*, vol. 31, no. 1, pp. 211–218, Feb. 2008.

[12] N. Ambasana, G. Anand, B. Mutnury, and D. Gope, "Eye height/width prediction from S-parameters using learning-based models," *IEEE Trans. Compon., Packag., Manuf. Technol.*, vol. 6, no. 6, pp. 873–885, Jun. 2016.

[13] H. Kim, C. Sui, K. Cai, B. Sen, and J. Fan, "Fast and precise high-speed channel modeling and optimization technique based on machine learning," *IEEE Trans. Electromagn. Compat.*, vol. 60, no. 6, pp. 2049–2052, Dec. 2018.

[14] T. Nguyen, T. Lu, J. Sun, Q. Le, K. We, and J. Schut-Aine, "Transient simulation for high-speed channels with recurrent neural network," in *Proc. 27rd IEEE Conf. Elect. Perform. Electron. Packag. Syst.*, Oct. 2018, pp. 303–305.

[15] K. S. Ooi, C. L. Kong, C. H. Goay, N. S. Ahmad, and P. Goh, "Crosstalk modeling in high-speed transmission lines by multilayer perceptron neural networks," *Neural Comput. Appl.*, to be published. [Online]. Available: https://link.springer.com/article/10.1007/s00521-019-04252-3. doi: 10.1007/s00521-019-04252-3.

[16] C. K. Ku, C. H. Goay, N. S. Ahmad, and P. Goh, "Jitter decomposition of high-speed data signals from jitter histograms with a pole-residue representation using multilayer perceptron neural networks," *IEEE Trans. Electromagn. Compat.*, to be published, doi: 10.1109/TEMC.2019.2936000.

[17] V. K. Devabhaktuni and Q.-J. Zhang, "Neural network training-driven adaptive sampling algorithm for microwave modeling," in *Proc. 30th Eur. Microw. Conf.*, Oct. 2000, pp. 1–4.

[18] C. H. Goay, P. Goh, N. S. Ahmad, and M. F. Ain, "Eye-height/width prediction using artificial neural networks from S-parameters with vector fitting," *J. Eng. Sci. Technol.*, vol. 13, no. 3, pp. 625–639, Mar. 2018.

[19] N. Ambasana, G. Anand, D. Gope, and B. Mutnury, "S-parameter and frequency identification method for ANN-based eye-height/width prediction," *IEEE Trans. Compon., Packag., Manuf. Technol.*, vol. 7, no. 5, pp. 698–709, May 2017.

[20] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Netw.*, vol. 2, no. 5, pp. 359–366, 1989.

[21] S. Haykin, *Neural Networks: A Comprehensive Foundation*, 2nd ed. Upper Saddle River, NJ, USA: Prentice-Hall, 1998.

[22] S. Koziel, X.-S. Yang, and Q.-J. Zhang, *Simulation-Driven Design Optimization and Modeling for Micorwave Engineering*. London, U.K.: Imperial College Press, 2013.

[23] C. A. Doukim, J. A. Dargham, and A. Chekima, "Finding the number of hidden neurons for an MLP neural network using coarse to fine search technique," in *Proc. 10th Int. Conf. Inf. Sci., Signal Process. Appl. (ISSPA)*, May 2010, pp. 606–609.

[24] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian optimization of machine learning algorithms," in *Proc. Neural Inf. Process. Syst.*, 2012, pp. 1–9.

[25] M. T. Hagan and M. B. Menhaj, "Training feedforward networks with the Marquardt algorithm," *IEEE Trans. Neural Netw.*, vol. 5, no. 6, pp. 989–993, Nov. 1994.

Authorized licensed use limited to: TRINITY COLLEGE LIBRARY DUBLIN. Downloaded on March 09,2026 at 14:30:17 UTC from IEEE Xplore. Restrictions apply.

[26] N. Ambasana, D. Gope, B. Mutnury, and G. Anand, "Application of artificial neural networks for eye-height/width prediction from S-parameters," in *Proc. IEEE 23rd Conf. Elect. Perform. Electron. Packag. Syst.*, Oct. 2014, pp. 99–102.
[27] W. T. Beyene, "Application of artificial neural networks to statistical analysis and nonlinear modeling of high-speed interconnect systems," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.,*, vol. 26, no. 1, pp. 166–176, Jan. 2007.
[28] V. K. Devabhaktuni, M. C. E. Yagoub, and Q.-J. Zhang, "A robust algorithm for automatic development of neural-network models for microwave applications," *IEEE Trans. Microw. Theory Techn.*, vol. 49, no. 12, pp. 2282–2291, Dec. 2001.
[29] W. Na and Q.-J. Zhang, "Automated parametric modeling of microwave components using combined neural network and interpolation techniques," in *IEEE MTT-S Int. Microw. Symp. Dig.*, Seattle, WA, USA, Jun. 2013, pp. 1–3.
[30] W. Na and Q.-J. Zhang, "Automated knowledge-based neural network modeling for microwave applications," *IEEE Microw. Wireless Compon. Lett.*, vol. 24, no. 7, pp. 499–501, Jul. 2014.
[31] W. Na and Q.-J. Zhang, "Unified automated knowledge-based neural network modeling for microwave devices," in *IEEE MTT-S Int. Microw. Symp. Dig.*, Aug. 2015, pp. 1–3.
[32] N. Ambasana, D. Gope, B. Mutnury, and G. Anand, "Eye-height/width prediction from S-parameters using bounded size training set for ANN," in *Proc. IEEE-EDAPS*, Bengaluru, India, Dec. 2014, pp. 17–20.
[33] M. Cavazzuti, *Optimization Methods: From Theory to Design*. Heidelberg, Germany: Springer-Verlag, 2013.
[34] J. E. Rayas-Sanchez and V. Gutierrez-Ayala, "EM-based statistical analysis and yield estimation using linear-input and neural-output space mapping," in *IEEE MTT-S Int. Microw. Symp. Dig.*, Jun. 2006, pp. 1597–1600.
[35] G. Brown, A. Pocock, M.-J. Zhao, and M. Luján, "Conditional likelihood maximisation: A unifying framework for information theoretic feature selection," *J. Mach. Learn. Res.*, vol. 13, pp. 27–66, Jan. 2012.
[36] E. Kiktova, M. Lojka, J. Juhar, and A. Cizmar, "Comparison of feature selection algorithms for acoustic event detection system," in *Proc. ELMAR*, Sep. 2014, pp. 1–4.
[37] A. C. Pocock, "Feature selection via joint likelihood," M.S. thesis, Dept. Comput. Sci., Univ. Manchester, Manchester, U.K., 2012.
[38] T. Kohonen, "The self-organizing map," *Proc. IEEE*, vol. 78, no. 9, pp. 1464–1480, Sep. 1990.
[39] M. D. McKay, R. J. Beckman, and W. J. Conover, "A comparison of three methods for selecting values of input variables in the analysis of output from a computer code," *Technometrics*, vol. 42, no. 1, pp. 55–61, 2000.
[40] P. Baldi, "Autoencoders, unsupervised learning, and deep architectures," in *Proc. ICML Workshop Unsupervised Transf. Learn.*, Jul. 2012, pp. 37–49.
[41] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *Proc. 2nd Int. Conf. Learn. Represent. (ICLR)*, May 2013, pp. 1–14.
[42] F. P. Casale, A. V. Dalca, L. Saglietti, J. Listgarten, and N. Fusi, "Gaussian process prior variational autoencoders," in *Proc. 32nd Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 10390–10401.

Chan Hong Goay was born in Bukit Mertajam, Malaysia, in 1992. He received the B.Eng. and M.Eng. degrees from Universiti Sains Malaysia, Nibong Tebal, Malaysia, in 2016 and 2019, respectively, where he is currently pursuing the Ph.D. degree with the School of Electrical and Electronic Engineering.

He is also a Graduate Research Assistant with the School of Electrical and Electronic Engineering, Universiti Sains Malaysia. His current research interests include artificial intelligence-based circuit modeling and signal integrity.

Azniza Abd Aziz (M'18–SM'19) received the Ph.D. degree from the University of South Carolina, Columbia, SC, USA.

She was an Advanced Signal Integrity Engineer with Intel Corporation, Penang, Malaysia, and a Senior Signal Integrity Engineer with Hewlett Packard Enterprise, Palo Alto, CA, USA, with ten years of experience in designing and validating desktop, mobile, and server platforms. She is currently with the School of Electrical and Electronic Engineering, Universiti Sains Malaysia, Nibong Tebal, Malaysia. Her current research interests include signal integrity solutions for high-speed data design, electromagnetic, communication systems, machine learning, and RF and microwave engineering.

Nur Syazreen Ahmad (M'14) was born in Kuala Lumpur, Malaysia. She received the B.Eng. degree (Hons.) in electrical and electronic engineering and the Ph.D. degree in control systems from The University of Manchester, Manchester, U.K., in 2009 and 2012, respectively.

Since 2013, she has been with the School of Electrical and Electronic Engineering, Universiti Sains Malaysia, Nibong Tebal, Malaysia. Her current research interests include motion control, robust stability, and performance analysis of constrained and nonlinear systems, optimization-based controller synthesis with linear matrix inequality searches, phase-locked loops, embedded control systems, robust control analysis, and design of autonomous mobile systems.

Dr. Ahmad became a member of the IEEE Control Systems Society in 2014.

Patrick Goh received the B.S., M.S., and Ph.D. degrees in electrical engineering from the University of Illinois at Urbana–Champaign, Champaign, IL, USA, in 2007, 2009, and 2012, respectively.

Since 2012, he has been with the School of Electrical and Electronic Engineering, Universiti Sains Malaysia, Nibong Tebal, Malaysia, where he currently specializes in the study of signal integrity for high-speed digital designs. His current research interests include the development of circuit simulation algorithms for computer-aided design tools.

Dr. Goh was a recipient of the Raj Mittra Award in 2012 and the Harold L. Olesen Award in 2010. He has served on the technical program committee and the international program committee in various IEEE and non-IEEE conferences around the world.

Authorized licensed use limited to: TRINITY COLLEGE LIBRARY DUBLIN. Downloaded on March 09,2026 at 14:30:17 UTC from IEEE Xplore. Restrictions apply.